30/4

Security

- symmetric of private key encryption, same key for encryption/decryption or different but related to encryption/decryption? -> same
- asymmetric of public key encryption, same key for encryption/decryption or different but related to encryption/decryption? -> different
- Andrew FS
  - secure link : users in campus
  - insecure link:
  - username/password: exposes to frequently -> MIM attack
  - why use symmetric private key encryption (extent login, performance,...)?
    - for performance, because symmetric is faster then asymmetric
  - core principle: authenticate user, prevent replay attack,...
  - in order to establish rpc session, the bind call in cleartext have to send client id, and encrypt a random number with a key and send to server, ... server

increment x+1, and send y random number back to client, what does the client know?
    - client knows the server is genuine + server knows the client is genuine
    - client's key might be compromised
  - why use session key?
    - to not overexposed hsk key
  - Why AFS use symmetric private key?
    - if use public key encryption -> N^2 key-pairs for each user and distribute to all workstations on campus
  - If use public key encryption, do need to send user id on clear text? -> no because public key can be used to identify user
  -

Quicksilver

- developed by IBM - 1985
- recovery should be backed into design
- distributed {client, server}
- QS bundles IPC with recovery management using lightweight txn
- a shadow graph structure emerges from client-server interactions
  - transaction tree root = owner + participants
  - Does transaction build-up cause overhead in communication?
    - client-server can choose to use the recovery management
  - Computational overhead and not communication overhead because communication is piggybacked going from one node to another (gonna happen anyway)
    - txn managers on different nodes communicate with no extra overhead as communication is piggybacked on IPC
  - Purpose of transaction? (db, recovery?) -> recovery management
  - One for entire DS or one for each sequence of server-client communication? -> for each server-client communication
  - transaction manager on behalf of clients do log for each client -> forcing to disk (recovery)
    - manager for all server-client interactions on its node
    - open file, talking to window manger for display ... (manger have to log)
  - Log management is common or distinct for each application? -> all applications

        - recovery management is built into OS, common for all applications -> careful forcing log onto disk, if fsync on a node -> all logs in memory will be pushed to disk -> can hurt performance
    - Compare QS recovery (all applications) vs LRVM recovery (all applications)
        - both are truth, if applications desire then they can have it
        - QS: applications ask QS
        - LRVM: applications can use LRVM library
    - QS vs LRVM, which is more comprehensive? -> QS is more comprehensive
        - any use system resource -> QS will do everything
            - malloc
            - open window manager ==> all have breadcrumbs (buggy software, system crashes, resource leak), QS try to recover everything? -> HOW?
        - LRVM only for memory
    - QS vs LRVM, which has more implementation overhead? -> depends
        - LRVM need set-range -> but might have no changes
        - LRVM use redo log might be created even if no changes are made
        - QS with help of devs, logging only log in memory, QS can do less work while LRVM have to log everything within set-range
        - QS might have to log everything from network, resource -> depends on volume
    - When is txn abort in QS?
        - not aborted at the first indication of failure# 22/4
        - allowing error reporting to continue and partial failures to be cleaned up when the coordinator initiates termination

22/4

LRVM

- LRVM reading
- applications require persistent
    - does not stop with just running process
    - beyond power failure
    - memory is volatile
    - fault tolerance

- file system = persistent data structure
- LRVM -> provide data abstraction -> help write application that requires persistent
    - begin ... end transaction, range of VA (of data structure to be modified)
    - HOW? range of addresses, using undo record incase of rollback
    - commit point -> create a redo log -> persistent storage
    - undo record vs redo log ?
        - undo only during txn
        - redo log is persisted
    - where is undo record? -> memory, only during txn
    - updates to metadata, data structure, is it going to llvm? -> doesnt go to LRVM, go directly into memory to modify using set range
    - at the end of txn, llvm creates a redo log in memory -> flush to disk
        - LLVM synchronously flush to disk? depends (can be no flush), can be parallel (trade of for risk)
    - open a file -> inode into memory by fs -> modify inode in memory -> have to flush do disk -> use LRVM to map inode to VA -> change to VA -> create redo log (change of data structure in memory result in redo log)
        - if didnt have LRVM -> make sure inode memory = inode in disk
    - LRVM txn vs DB txn? LRVM does not support ACID
        - DB txn: multiple txn (concurrent txn) -> need isolation between processes
        - LRVM baked into file server application -> consistency for single application, does not interact with other applications that use LVRM
            - multiple processes? responsibility of developers not LRVM
            - atomicity + durability
    - BEGIN TXN -> create undo record -> END TXN -> create redo log -> log apply to database (truncation)
        - In critical path, how many copies?
            1. undo record
            2. virtual memory changes for redo log in memory
            3. take redo log and put on the disk (can delay for not using log flush)
    - Recovery:
        - LRVM read from tail -> head
        - Why start from tail and not head?
            - reduce the work
            - LRVM reads the log of modified of uncommitted txn from the tail
            - read from head will see redundant work (changes to some record happens all over again)

        - log record {range, data}: {1-10: data} -> {1-10: data2} -> {1-10: data3}

RioVista

- provides LRVM semantic on top of RIO file cache
- software failure happens more frequent
- assume power failure is non issue
- file cache is battery-backed -> written into disk for recovery purpose
- using mmap file into memory
    - doesnt need to do fsync
- dont need synchronously write to disk + write back of file to disk can be delay
- only a portion of DRAM can have battery-backed
- an app using mmap ontop of Rio, does it need msync? -> NO, mmap ontop of file cached which is batter-packed -> doesnt need to worry about persistent
- How does delay writes from a file cache to the disk help?
    - procasinate help batch IO + reduce amount of IO
    - lots of files created only temporary and will be deleted -> no urgency to write to the disk
- OS does a lot of PROCASCINATION
- RioVista:
    - data segment map into VA
    - begin_transaction -> write change -> end_transaction -> get rid of undo log
    - What happens if there is an ABORT?
        - UNDO LOG: at start of transaction is persisted
        - ABORT: apply UNDO LOG to memory to recover old image because UNDO LOG services crashes
    - What are the implications of RioVista design? (no sync IO, no redo log, data segment directly modified)
        - all of the above
        - no redo log: write directly, only need UNDO log for abort
    - read, write during a txn, is it via file cache? -> YES, so that data is persisted
        - making changes directly on to data segment
        - which is mapped into VA
        - any change is persistent because of battery-backed
- memory pressure? conviction policy?
    - only a portion of memory is battery-backed

17/4

- latency: timer (hardware), preempt, scheudling
- periodic timer:
  - not real-time
  - latency of timer-event (need to be time-aware)
  - periodically sample
- oneshot timer:
  - pros: exact time
  - cons: os will be interrupted -> overhead
- soft timer:-
  - pros: reduce overhead, no additional interrupt to processor, OS will look for events to react to, polling
  - cons: latency + not timely, polling overhead
- firm timer: timely + reduce overhead
- APIC timer (advance programmable interrupt control)
- oneshot: timer goes off when the value reach 0
- overhead of oneshot timer?
- CPU get interrupted -> hurts performance of real-time sensitive applications -> avoid by scheduling the oneshot timer preceding the periodic timer -> avoid overhead of extra interrupt by the oneshot timer
- timer expired -> check to see if there is any timer will go off in the near future using OVERSHOOT DISTANCE
- can piggybacked on system call to go into the kernel to query if the one shot timer will go off
- exploit soft-timer + periodic timer => reduce overhead of oneshot timer + getting precise timing
- preempt latency
  - lock breaking kernel: explicit preemption point + when kernel not wokring on shared data structure
  - hierachial lock cannot be break into parts
- scheduling latency
  - proportion queue: every periodic T, program need a guarantee to get 1/3 of the period
  - priority inversion: make the lower priorty server same priority as the high level caller
- large scale situation awareness

- sending data overhead, false positive,...
- MapReduce: simplicity, scaling is automatic
- simpicity -> ease of use
- PTS:
    - propagate events
    - dealing with live + historical data uniformly with simple interface
    - PTS programming model: channel = commincation among entities
    - output + timestamp -> into channel
    - channel organized by timestamp
    - another computation want to get item from channel n ... n+1
        - can retrieve all items from lower bound and upper bound
        - process items then put into another channel with the same timestamp -> chain into a pipeline
    - Similarity between PTS channel vs Unix Socket? (unix abstraction, many to many, timestamp)
        - Unix abstraction: channel name unique same as socket name
    - Difference between PTS channel vs Unix Socket?
        - PTS has timestamp metadata within message, socket does not
        - Socket abstraction is 1-1 server-client
        - PTS channel (M-M) multiple threads -> one channel, 1 channel -> multiple threads
    - causuality is maintained using timestamp
    - when multiple threads output to same channel, does developers need to worry about mutual exclusion? -> NO, PTS abstraction all the synchronization,
        - item has timestamp -> cannot mutate
        - all items are stored in sequence because of timestamp -> no problem with mutual exclusion
    - relate time Lamport?
        - Lamport: logical + physical time
        - PTS only uses physical time, cameras have different timestamp, is there a global synchronization of clock?
            - ETP: encrypt time protocol -> roughly in-sync
        - PTS: the granularity of correctnes of ETP is enough, dont need to be globally synchronization
    - simplicity with put/get
    - time variables (new, old, latest,...)
    - two important concept

            - partial synchronisation causality: no inconsistency on shared data structure
                - getting mutual exclusion lock on shared data structure -> one thread will get the lock and can modif
                - time does not come into picture
                - correctness cannot come from partial order of which the data structure is modified
                - order of threads getting the lock that will be handle cannot be the property that application demands
            - temporal causality?
        - delivery system does the heavy lifting for streaming, storage,...
        - garbage collector or store ?

08/04

DQ

- given a capacity, DQ is constant, can increae Q and decrease D
- as a system admin, if can increase capacity -> can increase Q or D
- replication vs partitioning
    - replication
        - all servers have copies of all dataset
        - harvest can remain unchange -> yield will decrease > Q will go down
    - parition
        - failure -> full data not avialable -> harvest will surfer, yield remains unchanged
        - give parallelism but also want replication for full harvest (gmail,...)
        - some applications can deal with partial result (search,...)
- GMAIL: SAAS, implementation may change, API remains the same -> interact the same way
- Do Google Search, what happens? -> Computation running parallel on large data center

MapReduce

- need coordination between the parallelism happens
- resources available on the cloud -> help with developers -> painless for domain experts
- Why map-reduce?
    - steps in processing can be implementing using map-reduce
    - most usecase need a map function and a reduce function
    - heavely lifting, number of mappers + reducers + coordination done by map-reduce framework
    - page rank: map will transform url, reduce: aggregate the occurence
- In case of failure, the number of maps? -> numbers of shard, 10 shards -> 10 mappers
- In case of failure, the number of reducers? -> numbers of distinct output
- How many intermediate files that will be passed from mappers to reducers? -> M immediate files, R reducers -> M * R
- When does the reducers start reading the file? -> when the mappers is done
- reducer finishes and produces an output file (temp file as first), making the file visible to user is job of master -> use rename to make visible to user -> the stragger will be ignored

CDN

- DHT
    - implementation for CDNs to populate the routing table at the user level
    - PUT <key,value>, GET KEY -> value
    - Traditional greedy approach
        - key value is placed in a node that is close to the key
        - get to destination with minimum hops
        - if go directly -> overload at the destination, tree saturation where nodes in proximity to the congested node also become congested
    - server overload -> mirror content at geo-locals sites => expensive
- CORAL
    - sloppy DHT spreads metadata
    - distance is computed by using XOR the bit patterns of the node IDs for the source and destination
    - go to half the distance at every hop in the node ID namespace
        - if the node does not have a direct way to reach the desired node, a nearby node is contracted to obtain information on nodes that are close

enough to the destination
        - reduces the distance by half to find the appropriate node to place the key
        - asked each node along the way if it is loaded or full
        - if full, retracted to choose an appropriate node
    - reduce congestion in the network
    - dealing at application level where content is distributed
    - Operation
        - PUT: <key, value>
            - key: is the content hash, value is node id of the proxy with the content
            - place the key in an appropriate node based on space and time metrics
            - FULL: already storing value for a particular key
            - LOADED: how many requests per unit time a node is willing to store a particular key
        - GET
    - what happens if the node is "full" for a specify key?
        - during the forward phase of the put operation
        - each hop checks whether the node is full or loaded for that key
        - if node is full, the system assumes tree saturation meaning nodes closer to that node also likely full
        - the algorithm stop processing toward the destination and instead store the key at earlier node
    -

01/04

- NFS remains centralized in its management of files -> unbalanced server loads
- XFS decouple metadata management from data storage
- both aim to mitigate disk latency
    - LFS: treating the entire file system as continuous log
    - JFS: uses temporary logs to update data files
- Cooperate caching
    - XFS: peer-to-peer caching model where clients server as both consumer + providers of data
        - aggregate memory of local network -> minimize disk access

        - static assignment at file creation to maintain scalability through global replication
- NFS
    - centralized management: each file partition is managed by a dedicated server
    - lopsided loading: a hot file can overwhelm a single server while other servers remain idle
    - static association: no mechanism to move metadata management to an idle server to balance load
- Journaling file system (JFS)
    - log files + data files
    - read: standard file access
    - logs are temporary, applied to data files, discard log files
    - reduces disk by batching changes
- Log-structured file system (LFS)
    - only log files
    - logs are persistent and represent the current state
    - read: data must be reconstructed from various log segments
    - amortizes disk by writing large, contiguous log segments
- XFS: files do not exist on disk, system uses Logs Segments
    - m map: maps an index number -> specific metadata manager node
        - globally replicated
        - statically assigned at creation
    - file dir: maps human-readable file to index number
        - on the client node, where the file was created
    - imap: maps i-number to inode
        - partitioned among metadata managers
    - inode: pointers -> disk address of the log segments
        - one per file
    - stripe group map: map log segments IDs -> set of storage server
        - fault tolerance
        - bandwidth
        - globally replicated
    - data aggregation
        - changes to multiple files are recorded contiguously in a log segment
        - flush when full in memory
    - network stripping: similar to RAID but for network
    - periodically clean the log + coalesces active data into new segments to reclaim space

    - fastest path - local cache: accesses a file it recently created or read directly from local memory
    - 2nd best: peer/cooperative cache
        - if data is not local -> metadata manager identifies peer that has the file in its memory
        - data is served over network -> reduce disk io
    - longest path:
        - query M map to find manager
        - manager consults the IMAP to find the I-node location
        - I-node identifies the necessary Log Segment IDs
        - the stripe group map identifies which storage servers hold the stripes of the segments
        - the system retrieves the stripes -> reconstruct the log segments -> extracts the requested data blocks
    - Trade-offs
        - granularity: choose block sizes and log segments size carefully
            - large blocks cause internal fragmentation
            - small blocks increase the complexity of the metadata
        - fault tolerance
            - striping log segment across network -> window of vulnerability
            - node crashes before flush -> data is lost
        - static vs dynamic management
            - DN is ideal for load balancing
            - simplifies by assigning managers statically at the creation time

25/03

- Release Consistency (RC)
    - RC recognizes that parallel programs typically use synchronization primitives (acquire/release) to govern access to shared data.
    - Mechanism: Coherence actions are only required to be complete at the point of a "release" operation.
    - Synchronization Labels: Accesses are explicitly labeled as Acquire
    - Barrier Synchronization: This is a special case where "acquire" and "release" operations are effectively rolled into a single operation (entering and exiting the barrier).
- Eager vs Lazy Release Consistency

    - two primary models for implementing release consistency:
        - Eager Release Consistency (ERC) -> "push model," system broadcasts all changes to all peers at point of release because it does not know which node will next acquire the lock -> higher message traffic but lower latency upon acquisition
        - Lazy Release Consistency (LRC) -> "pull model" characterized by procrastination, coherence actions delayed until specific node performs acquisition -> reduces number of messages but may increase latency at point of acquisition
- Software DSM Implementation: Treadmarks
    - Implementing DSM in software requires bridging the gap between the application's view of memory and the operating system's management of physical resources.
    - The Feasibility of Fine-Grain Sharing
    - Page Fault Handling Mechanism
- Fault Detection: The OS VM manager detects a page fault when a thread accesses a virtual address not mapped to physical memory.
- DSM Intervention: The OS hands the fault to the DSM software.
- Peer Communication: The DSM software identifies the owner of the page (statically determined for simplicity) and retrieves the current copy across the network.
- Resumption: The page is populated into physical memory, and the OS resumes the faulting thread.
- The Multi-Writer Protocol
    - Earlier DSM systems used a "single-writer" protocol, which caused false sharing: a situation where the page "ping-pongs" between nodes because different threads are writing to different, unrelated data structures that happen to reside on the same page.
    - Treadmarks solved this through a multi-writer approach:
        - Twinning: Before a page is modified, the system creates a "twin" (a pristine copy).
    - Diffing: At the point of release, the system compares the modified page to the twin to create a run-length encoded diff (recording only the specific bytes changed).
        - Synchronization Causality: Diffs are applied to the pristine copy in the order they were created, managed via Vector Timestamps. This allows multiple nodes to write to different parts of the same page concurrently without conflict.
- The Evolution to Structured DSM

    - "unstructured" DSM—the attempt to make a linear, flat address space appear shared—has largely faded from use due to scalability issues
    - Current State -> Modern distributed systems utilize Structured DSM
    - Rationale -> Applications generally require data structures rather than raw linear memory, Structured DSM provides shared key space that is easier to partition and scale across massive clusters

17/03

- Spring Kernel (developed at Sun Microsystems, architected by Ysef Khaledi) -> landmark in distributed operating system design with primary goals of extensibility and openness for third-party developers to integrate software and device drivers
    - Microkernel Minimalism
        - Following "Least Case Principle," Spring microkernel kept intentionally small:
            - The Nucleus -> manages threads and Inter-Process Communication (IPC)
            - Address Space Management -> manages basic virtual memory framework
        - Everything else (network proxies, device drivers) resides outside kernel -> open, flexible framework where system services implemented in protected domains and accessed via object invocation
    - Inter-Domain Communication
        - Spring utilizes "Door" abstraction and corresponding "Door Table" (analogous to Unix file descriptors):
            - process opens door -> gains capability for cross-domain calls
            - OS maintains door table per process -> tracks these capabilities
            - all system services accomplished through object invocation across protection domains
- Spring vs. Tornado
    - Critical distinction in how different operating systems employ object technology
    - Primary Goal
        - Spring Kernel -> modularity, specialization, and integration of third-party code
        - Tornado -> incremental performance optimization of system services
    - Visibility

        - Spring Kernel -> objects externally visible, others can write to and integrate with them
        - Tornado -> clustered objects are internal optimizations "buried" inside kernel
    - Role
        - Spring Kernel -> basic system-structuring mechanism
        - Tornado -> selective optimization tool, system services remain unchanged
    - Spring provides sophisticated users flexibility to define their own memory management policies through tiered abstraction model
    - Memory Abstractions
        - Regions -> linear address space for process broken into sets of pages
        - Memory Objects -> abstractions that map to physical entities like files or swap space
        - Pager Objects -> handle page faults and manage mapping between regions and memory objects
    - Consistency and Policy Flexibility
        - Spring allows multiple pagers to coexist for different regions of same address space
            - Consistency Responsibility -> if shared pages exist across different Virtual Memory Managers (VMMs), responsibility for ensuring consistency lies with pager objects, not Spring kernel
            - Developer Freedom -> openness allows developers to implement specialized paging algorithms (high-performance databases) without compromising kernel's integrity
    - Spring pioneered client-server paradigm for distributed services using "secret sauce" known as Subcontract
        - Subcontract offloads connectivity logic from application, decides at runtime how client should communicate with server:
            - Local Communication -> client and server on same machine, subcontract may use shared memory
            - Remote Communication -> client and server on different machines, subcontract handles argument marshaling and chooses appropriate transport protocol (UDP for LANs or TCP/IP for congested networks)
            - Transparency -> client oblivious to whether talking to local server, replicated server, or cache
        - Network proxies in Spring exist entirely outside kernel -> when client makes remote call, interacts with local proxy object, which communicates

with peer proxy on remote node via subcontract-selected protocol to pass invocation to server domain

- Java's Distributed Object Model
    - Remote Method Invocation (RMI) -> fundamental building block for Java's distributed objects
    - Remote Reference Layer (RRL) -> functionally similar to Spring's subcontract mechanism, manages connection setup, liveness monitoring, and transport choice (UDP vs. TCP/IP) based on location of endpoints
    - Parameter Passing -> unlike local Java objects where references passed, distributed model requires parameters sent as value-results when crossing Virtual Machine boundaries

10/03

- Latency vs. Throughput
    - Latency -> elapsed time for single event, focused on individual transaction speed
    - Throughput -> number of events completed per unit time, driven by parallelism and bandwidth
    - Illustrative Example -> person walking from parking lot to classroom represents latency (mins), if second person walks with them, latency remains same but throughput doubles
- Optimizing Remote Procedure Call (RPC) Performance
    - RPC -> building block of client-server paradigm, performance optimization focuses on reducing software overhead (hardware overhead/NIC bandwidth is fixed)
    - Reducing Data Copying Overhead
        - Traditional RPC involves copies on client side
            - Client Stub -> creating RPC packet at user level
            - Kernel Buffer -> transferring packet from user space to kernel
            - Network Transfer -> moving data from kernel buffer via DMA to network
        - Two primary options to reduce copies:
            - Option A (Stub in Kernel) -> dropping client stub into kernel allows direct arguments from stack, drawback: increases kernel's attack

surface and security vulnerabilities
        - Option B (Shared Descriptors) -> utilizing shared descriptor (similar to Zen's ring buffer) between client stub and kernel, descriptor contains pointers to arguments in client stack -> kernel copies data directly, superior method: efficiency + security
    - Control Transfers and Context Switches
        - Standard RPC involves context switches
            - Client-side -> switching from calling process to another process to keep CPU busy during call
            - Server-side -> switching to server process upon packet arrival
            - Server-side -> switching away from server process after completion to maintain CPU utilization
            - Client-side -> switching back to original client process to receive result
        - Optimization Strategies
            - Critical Path -> only switches are in critical path of RPC latency, others can be overlapped with network communication
            - Spinning vs. Switching -> reduce latency by spinning (busy-waiting) on client side for short duration rather than context switching, if response doesn't arrive quickly then perform switch, best of both worlds approach: avoids cache pollution and context switch overhead for fast RPCs

- Active Networks and ANTS Toolkit
    - Active Network experiment -> circa mid-90s, sought to provide Quality of Service (QoS) guarantees (essential for jitter-free video streaming) by making network programmable
    - Capsules and Type Fields
        - ANTS toolkit converts application payloads into capsules
            - Structure -> capsule contains ANTS header and payload, wrapped in standard IP header for compatibility with existing routers
            - Type Field -> header includes type field (unique fingerprint/hash) of code required to process packet
    - Code Distribution and Execution
        - When active node (programmable switch) receives capsule
            - checks soft store (cache) for code matching type fingerprint

        - if code missing -> requests code from previous node in transmission path
        - if previous node lacks code (due to cache replacement) -> packet dropped
        - Rationale -> packet loss is standard in networking, if code gone then flow likely inactive, higher-level protocols (like TCP) handle retransmission if data still required
    - Security and Evolution
        - ANTS uses Java-based sandbox to isolate flows, mitigate risks from executing foreign code
        - Active networks had no killer app in 90s, concept revitalized as Software-Defined Networking (SDN)
        - SDN now used extensively in cloud data centers (Google, Meta) -> virtualize network traffic and ensure isolation between different tenants

- Ensemble System: Component-Based Design
    - Inspired by VLSI in hardware -> chips built from Lego-like blocks, Ensemble project at Cornell applied modularity to software protocol stacks
    - Ensemble Design Cycle
        - bridges gap between high-level specification and efficient deployment through cycle
            - Specification -> using IO Automatato define system goals and verify logic
            - Implementation -> coding in ocaml (object-oriented language with well-defined interfaces, avoids side effects common in C)
            - Optimization -> translating ocaml code into New Pearl (theoretical framework), Theorem Prover optimizes common case (sequential packet arrival) and produces verified, functionally equivalent version
    - Performance and Reliability
        - Ensemble proved modular protocol stack could perform well as monolithic system
        - Modularity aids in debugging -> best way to debug program is write it again, if build code out of modular components then component small enough to throw away and rewrite
    - Kernel Modules -> modern Linux capabilities and User Space File Systems (FUSE) are evolutions of ideas from early RPC and kernel-stub research

    - Microkernels -> practical to build high-performance servers (like HTTP server) on top of microkernels by applying optimization strategies
    - Shoulders of Giants -> current developments in cloud computing and networking built upon foundational thought experiments from mid-90s

26/02

- Fundamentals of Distributed Systems
    - Distributed system defined by nodes connected via LAN or WAN, communication exclusively through messages
    - Message communication time significantly greater than local event time
    - Happened-Before Relationship
        - A -> B indicates event A happened before event B
        - Causality determined by:
            - Local Events -> if A and B on same node and A precedes B
            - Communication Events -> sending message must happen before receiving message
    - Lamport's Logical Clock Conditions
        - Rules applied:
            - Monotonic Increase -> on single machine, if B follows A, timestamp B > timestamp A
            - Send/Receive Integrity -> timestamp of sent message < timestamp of received message
            - Clock Update -> upon receiving message, assign timestamp Max(local, received) + 1
        - Concurrent Events -> if Clock(A) < Clock(B), does not mean A happened before B, could be concurrent
- Constructing Total Order and Determinism
    - Logical clocks provide partial order, some tasks require total order
    - Total Ordering -> constructed from partial order by breaking ties among concurrent events
    - Tie-Breaking -> arbitrary but consistent, smaller PID wins
    - Deterministic Algorithms -> built using logical clocks with consistent tie-breaking rules
- Distributed Mutual Exclusion Algorithm
    - Lamport's algorithm ensures only one node accesses shared resource at time
    - Process

        - Request -> node puts request in local queue, broadcasts request with timestamp to all other nodes
        - Acknowledgment -> other nodes acknowledge lock request
        - Entry Condition -> node believes it has lock if:
            - own lock request at head of local queue
            - received acknowledgments from all other nodes OR received lock requests from all other nodes with later timestamps
    - Essential Dependencies
        - Correctness hinges on three guarantees:
            - FIFO Ordering -> messages go in order between any two nodes
            - Reliability -> no loss of messages
            - Causality -> adheres to happened-before relationship
        - Out-of-Order Anomaly -> if messages arrive out of order, node might incorrectly assume it has lock, violating mutual exclusion
- Physical Clocks and Real-Time Anomalies
    - In scenarios with real-world assets, logical clocks insufficient
    - Physical clocks must be synchronized to prevent anomalies
    - Banking Example
        - Deposit at ATM X, withdraw at ATM Y -> if ATM Y clock lags, withdrawal might fail as bank sees withdrawal before deposit
    - Drift and Synchronization Constants
        - Individual Drift -> rate local clock drifts relative to perfect atomic clock
        - Mutual Drift -> maximum difference between clocks of any two nodes
        - IPC Lower Bound -> minimum time for inter-process communication
    - Physical Clock Condition
        - If met, lower bound of communication time exceeds potential drift, avoiding temporal anomalies

18/02

- Synchronization Barrier Algorithms
    - Primary challenge in barrier synchronization for parallel systems is managing race conditions and contention on shared variables
    - Centralized Barriers
        - Simple count-based barriers suffer from race conditions where early-arriving processors might exit barrier and re-enter next before last processor has reset count

        - Sense-Reversing Barrier solves by using flag to flip sense of barrier (zero vs one), ensuring processors only proceed when current phase complete
    - Hierarchical and Distributed Barriers
        - To scale beyond small number of processors, various hierarchical structures employed
        - Key Technical Nuances
            - MCS Barrier -> each processor spins on unique location, works on NCC NUMA because remote writes trigger local cache coherence within destination SMP node, informing waiting parent, does not require Mutual Exclusion because each shared variable has only one designated writer
            - Tournament Barrier -> algorithm versatile because arrows (signals) can be implemented as memory writes or network messages, preferred choice for cluster architectures
            - Dissemination Barrier -> unlike tree structures where communication decreases as you move up, dissemination involves N messages per round for log N rounds, higher communication overhead but increased robustness
- Lightweight Remote Procedure Calls (LRPC)
    - When services communicate across protection domains within same machine, overhead of traditional RPC prohibitive
    - The Overhead Problem
        - Standard RPC involves:
            - Traps -> two traps (call and return)
            - Context Switches -> two switches between client and server
            - Data Copying -> four copies per direction (Client stack -> Client stub buffer -> Kernel buffer -> Server domain buffer -> Server stack)
    - The LRPC Solution
        - LRPC reduces copying by establishing Astack (Argument Stack) as shared memory region between client and server
        - Copy Reduction -> number of copies drops to two (one for marshalling into Astack and one for unmarshalling)
        - Language Optimizations -> sophisticated languages like Modula-3 can reduce to one copy by swizzling stack, allowing server procedure to execute directly out of shared Astack
- Multiprocessor Scheduling and Cache Affinity
    - Effectiveness of multiprocessor scheduling largely determined by Cache Affinity —degree to which thread's working set remains in processor's cache

- Scheduling Policies
  - Thread-Centric -> focus on where thread last ran (Fixed Processor, Last Processor)
  - Processor-Centric -> focus on state of processor (Minimum Intervening Threads)
  - MIQ (Minimum Intervening + Queue) -> selects processor based on lowest number of intervening threads (preserve affinity) while accounting for length of ready queue (minimize wait time)
- Procrastination in Scheduling
  - Implementing Idle Loop can improve performance
  - By making processor wait briefly for high-affinity thread rather than immediately scheduling low-affinity thread, system avoids cache pollution and improves overall throughput
- Tornado: The Clustered Object Paradigm
  - Tornado designed for large-scale NUMA machines based on principle: Shared memory machines scale well when you don't share memory
  - Deconstruction of the Page Table -> instead of central page table, Tornado uses Regions, each region represents disjoint subset of virtual pages
  - Clustered Objects -> logical shared object (like process or region) may have multiple physical representations (replicated or partitioned) to minimize cross-node contention
  - Object Breakdown:
    - Process Object -> replicated per CPU (mostly read-only)
    - Region Object -> partially replicated and partitioned
    - DRAM (DM) Object -> replicated per NUMA node to ensure local memory allocation
- Corey: Application-Level Control
  - Corey extends Tornado's ideas by allowing sophisticated applications to manage own sharing
  - Through Address Ranges and Shares, applications explicitly announce intent to share data, allowing kernel to avoid unnecessary coherence maintenance
- Cellular Disco: Virtualizing Parallel I/O
  - Cellular Disco utilizes thin virtualization layer (hypervisor) to allow multiple guest operating systems to run on large parallel machine
  - Innovation -> delegates I/O to dormant host operating system (e.g., IRIX)

- Process -> when guest makes I/O request, Cellular Disco performs bookkeeping, updates interrupt vector, wakes host OS to execute driver, upon completion interrupt intercepted by Cellular Disco and passed back to guest, avoids need to rewrite complex device drivers for new experimental operating systems